Marlin PC 구현 상세 분석

개요

Marlin PC는 KZG10을 기반으로 하되, **degree bound 강제**와 **배치 처리** 기능을 추가한 다항식 커밋먼트 스킴입니다. 이 문서에서는 Marlin PC가 KZG10과 어떻게 다른지, 그리고 핵심 기능들이 어떻게 구현되어 있는지 상세히 분석합니다.

KZG10과 Marlin PC의 주요 차이점

1. Degree Bound 강제 메커니즘

KZG10의 한계:

- KZG10은 다항식의 degree bound를 강제하지 않습니다
- 커밋된 다항식이 실제로 선언된 차수 이하인지 검증할 수 없습니다

Marlin PC의 해결책:

- **Shifted Commitment** 기법을 사용하여 degree bound를 강제합니다
- 원본 다항식과 함께 “shifted” 버전의 다항식도 커밋합니다

```
/// Marlin PC의 Commitment 구조
pub struct Commitment<E: Pairing> {
/// 원본 다항식에 대한 KZG10 커밋
pub comm: kzg10::Commitment<E>,
/// Shifted 다항식에 대한 KZG10 커밋
/// degree bound가 없는 경우 None
```

```
pub  shifted_comm: Option<kzg10::Commitment<E>>,

}
```

2. Shifted Polynomial의 개념

Degree bound `d` 를 가진 다항식 `p(x)` 에 대해, shifted polynomial은 다음과 같이 정의됩니다:

```
shifted_p(x) = x^(max_degree - d) * p(x)
```

이렇게 하면:

- 원본 다항식 `p(x)` 의 차수가 `d` 이하라면, `shifted_p(x)` 의 차수는 정확히 `max_degree` 가 됩니다
- 만약 `p(x)` 의 차수가 `d` 를 초과한다면, `shifted_p(x)` 의 차수도 `max_degree` 를 초과하게 됩니다

```
/// Shifted polynomial 생성 함수

pub(crate) fn shift_polynomial<E: Pairing, P: DenseUVPolynomial<E::ScalarField>>

ck: &CommitterKey<E>,

p: &P,

degree_bound: usize,

) -> P {

if p.is_zero() {

P::zero()

} else {

let enforced_degree_bounds = ck
```

```
.enforced_degree_bounds

.as_ref()

.expect("Polynomial requires degree bounds, but `ck` does not support any");

let largest_enforced_degree_bound = enforced_degree_bounds.last().unwrap();



// x^(max_degree - degree_bound) 만큼 shift

let mut shifted_polynomial_coeffs =

vec![E::ScalarField::zero(); largest_enforced_degree_bound - degree_bound];

shifted_polynomial_coeffs.extend_from_slice(&p.coeffs());

P::from_coefficients_vec(shifted_polynomial_coeffs)

}

}
```

Marlin PC의 핵심 구성 요소

1. CommitterKey 구조

```
pub struct CommitterKey<E: Pairing> {

/// 일반 다항식 커밋을 위한 powers

pub powers: Vec<E::G1Affine>,



/// Shifted 다항식 커밋을 위한 powers

/// degree bound를 지원하지 않으면 None
```

```
pub shifted_powers: Option<Vec<E::G1Affine>>,


/// Hiding을 위한 powers
pub powers_of_gamma_g: Vec<E::G1Affine>,


/// 지원하는 degree bound들 (오름차순 정렬)
pub enforced_degree_bounds: Option<Vec<usize>>,
/// 최대 차수
pub max_degree: usize,
}
```

2. VerifierKey 구조

```
pub struct VerifierKey<E: Pairing> {
/// 기본 KZG10 검증 키
pub vk: kzg10::VerifierKey<E>,
/// Degree bound 강제를 위한 정보
/// (degree_bound, shift_power) 쌍들의 벡터
pub degree_bounds_and_shift_powers: Option<Vec<(usize, E::G1Affine)>>,
pub max_degree: usize,
pub supported_degree: usize,
}
```

Commit 과정 분석

Marlin PC의 commit 과정은 다음과 같습니다:

```
// 1. 원본 다항식에 대한 KZG10 커밋 생성
let (comm, rand) = kzg10::KZG10::commit(&ck.powers(), polynomial, hiding_bound,

// 2. Degree bound가 있는 경우 shifted commitment도 생성
let (shifted_comm, shifted_rand) = if let Some(degree_bound) = degree_bound {
let shifted_powers = ck.shifted_powers(degree_bound).ok_or(Error::UnsupportedDeg
let (shifted_comm, shifted_rand) = kzg10::KZG10::commit(&shifted_powers, &polyno
(Some(shifted_comm), Some(shifted_rand))
} else {
(None, None)
};

// 3. 최종 커밋 구성
let comm = Commitment { comm, shifted_comm };
let state = Randomness { rand, shifted_rand };
```

Linear Combination 처리

Marlin PC의 핵심 기능 중 하나는 여러 다항식의 선형 결합을 효율적으로 처리하는 것입니다.

1. Commitment 결합

```
/// 여러 커밋을 선형 결합으로 합치는 함수
fn combine_commitments<'a>(
coeffs_and_comms: impl IntoIterator<Item = (E::ScalarField, &'a marlin_pc::Commi
) -> (E::G1, Option<E::G1>) {
let mut combined_comm = E::G1::zero();
let mut combined_shifted_comm = None;
for (coeff, comm) in coeffs_and_comms {
// 원본 커밋 결합
if coeff.is_one() {
combined_comm.add_assign(&comm.comm.0);
} else {
combined_comm += &comm.comm.0.mul(coeff);
}

// Shifted 커밋 결합 (있는 경우)
if let Some(shifted_comm) = &comm.shifted_comm {
let cur = shifted_comm.0.mul(coeff);
combined_shifted_comm = Some(combined_shifted_comm.map_or(cur, |c| c + cur));
}
}
(combined_comm, combined_shifted_comm)
```

```
}
```

2. Challenge 기반 Accumulation

Marlin PC는 Fiat-Shamir 변환을 사용하여 여러 커밋과 값들을 하나로 accumulate합니다:

```
fn accumulate_commitments_and_values<'a>(
commitments: impl IntoIterator<Item = &'a LabeledCommitment<marlin_pc::Commitmen
values: impl IntoIterator<Item = E::ScalarField>,
sponge: &mut impl CryptographicSponge,
vk: Option<&marlin_pc::VerifierKey<E>>,
) -> Result<(E::G1, E::ScalarField), Error> {
let mut combined_comm = E::G1::zero();
let mut combined_value = E::ScalarField::zero();
for (labeled_commitment, value) in commitments.into_iter().zip(values) {
let degree_bound = labeled_commitment.degree_bound();
let commitment = labeled_commitment.commitment();
// 첫 번째 challenge 생성
let challenge_i = sponge.squeeze_field_elements_with_sizes(&[CHALLENGE_SIZE])[0]
// 원본 커밋과 값 accumulate
combined_comm += &commitment.comm.0.mul(challenge_i);
combined_value += &(value * &challenge_i);


// Degree bound가 있는 경우 shifted commitment 처리
```

```
if let Some(degree_bound) = degree_bound {
let challenge_i_1 = sponge.squeeze_field_elements_with_sizes(&[CHALLENGE_SIZE])[
let shifted_comm = commitment.shifted_comm.as_ref().unwrap().0.into_group();
let shift_power = vk.unwrap().get_shift_power(degree_bound)
.ok_or(Error::UnsupportedDegreeBound(degree_bound))?;


// Shifted commitment에서 shift_power * value를 빼고 challenge로 곱함
let mut adjusted_comm = shifted_comm - &shift_power.mul(value);
adjusted_comm *= challenge_i_1;
combined_comm += &adjusted_comm;
}
}
Ok((combined_comm, combined_value))
}
```

Verification 과정

1. 기본 검증

Marlin PC의 검증은 다음 단계로 이루어집니다:

1. **Commitment Accumulation**: 여러 커밋을 challenge를 사용해 하나로 합침
2. **KZG10 검증**: 합쳐진 커밋에 대해 기본 KZG10 검증 수행
3. **Degree Bound 검증**: Shifted commitment를 통해 차수 제한 확인

2. Batch Verification

```
fn batch_check<'a, R: RngCore>(
vk: &Self::VerifierKey,
commitments: impl IntoIterator<Item = &'a LabeledCommitment<Self::Commitment>>,
query_set: &QuerySet<P::Point>,
values: &Evaluations<E::ScalarField, P::Point>,
proof: &Self::BatchProof,
sponge: &mut impl CryptographicSponge,
rng: &mut R,
) -> Result<bool, Self::Error> {
// 1. 커밋들을 결합하고 정규화
let (combined_comms, combined_queries, combined_evals) =
Marlin::<E, P, Self>::combine_and_normalize(
commitments, query_set, values, sponge, Some(vk)
)?;
// 2. KZG10 batch check 수행
let result = kzg10::KZG10::batch_check(
&vk.vk,
&combined_comms,
&combined_queries,
&combined_evals,
&proof,
```

```
rng,

)?;

Ok(result)

}
```

성능 특성 및 최적화

1. Batch Operations의 이점

- **여러 증명을 동시에 검증**: 개별 검증보다 효율적
- **Challenge 재사용**: 동일한 challenge를 여러 커밋에 적용
- **MSM 최적화**: Multi-scalar multiplication을 배치로 처리

2. Memory 사용량

- **추가 저장공간**: Shifted commitment로 인해 약 2배의 저장공간 필요
- **Setup 복잡성**: Degree bound별로 추가 파라미터 생성 필요

3. 계산 복잡도

- **Commit**: O(d) + O(d) = O(d) (원본 + shifted)
- **Open**: O(d) + O(d) = O(d)
- **Verify**: O(1) (pairing 연산 횟수는 동일)

보안 고려사항

1. Degree Bound 강제의 중요성

- **Soundness**: 잘못된 차수의 다항식으로는 유효한 증명을 생성할 수 없음
- **Knowledge Soundness**: Prover는 실제로 올바른 차수의 다항식을 알고 있어야 함

2. Challenge 생성

- **Fiat-Shamir**: 암호학적으로 안전한 해시 함수 또는 sponge 사용 필수
- **Challenge 공간**: 충분히 큰 필드에서 challenge 생성

3. Trusted Setup

- **Universal Setup**: KZG10과 동일한 trusted setup 사용
- **Parameter 재사용**: 동일한 universal parameter로 다양한 degree bound 지원

실제 사용 예제

```
use ark_poly_commit::{marlin_pc::MarlinKZG10, PolynomialCommitment, LabeledPolyn

use ark_bls12_381::Bls12_381;

use ark_poly::univariate::DensePolynomial;


type PC = MarlinKZG10<Bls12_381, DensePolynomial<Fr>>;


// 1. Setup

let max_degree = 100;

let pp = PC::setup(max_degree, None, &mut rng)?;


// 2. Trim with degree bounds
```

```
let supported_degree = 50;

let hiding_bound = 10;

let enforced_degree_bounds = Some(&[20, 30, 40][..]);

let (ck, vk) = PC::trim(&pp, supported_degree, hiding_bound, enforced_degree_bou



// 3. Commit with degree bound

let poly = DensePolynomial::rand(25, &mut rng);

let labeled_poly = LabeledPolynomial::new("test".to_string(), poly, Some(30), So

let (commitments, states) = PC::commit(&ck, [&labeled_poly], Some(&mut rng))?;



// 4. Open and verify

let point = Fr::rand(&mut rng);

let proof = PC::open(&ck, [&labeled_poly], &commitments, &point, &mut sponge, &s

let value = labeled_poly.evaluate(&point);

let is_valid = PC::check(&vk, &commitments, &point, [value], &proof, &mut sponge
```

결론

Marlin PC는 KZG10의 효율성을 유지하면서도 degree bound 강제라는 중요한 기능을 추가한 발전된 다항식 커밋먼트 스킴입니다. Shifted commitment 기법과 challenge 기반 accumulation을 통해 안전하고 효율적인 배치 처리를 지원하며, 영지식 증명 시스템에서 핵심적인 역할을 수행합니다.

관련 문서

기반 기술

- **KZG10 구현 분석**: Marlin PC의 기반이 되는 KZG10 스킴의 상세 분석
- **핵심 인터페이스**: Marlin PC가 구현하는 공통 인터페이스들

비교 분석

- **기타 스킴들**: Sonic PC 등 Marlin PC와 유사한 다른 스킴들과의 비교

실습 및 참고

- **사용 예제**: Marlin PC의 degree bound 기능을 활용한 실제 사용 예제
- **암호학적 배경**: Degree bound 강제의 보안적 의미와 중요성
- **용어집**: Marlin PC 관련 용어와 개념